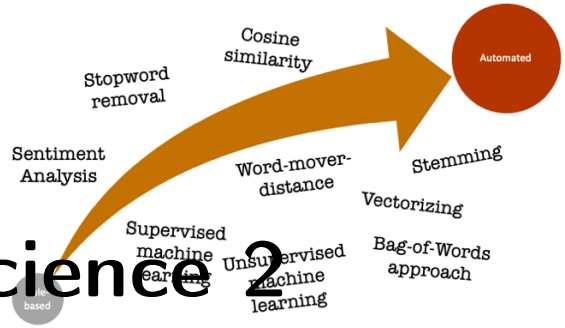


Computational Communication Science 2

Week 1 – Introduction & Working with Textual Data

Anne Kroon Saurabh Khanna

March 30, 2026

Digital Society Minor
University of Amsterdam

Today

- Course introduction
- What is Computational Communication Science?
- Why study text as data?
- Two ways to analyze text
- First steps in preprocessing

The people

Introducing. . . Saurabh

dr. Saurabh Khanna

Assistant Professor, Youth & Media Entertainment

- Studies how people understand information in a digital world
- Uses computational methods to study communication and media

| s.khanna@uva.nl | https://www.uva.nl/en/profile/k/h/s.khanna/s.khanna.html

Introducing. . . Anne

dr. Anne Kroon

Associate Professor, Communication, Organizations & Society

- Research on biased AI in recruitment
- Research on media bias regarding minorities
- Works with text analysis and word embeddings

@annekroon | a.c.kroon@uva.nl | http://www.uva.nl/profiel/k/r/a.c.kroon/a.c.kroon.html

Introducing... Roeland

Tutorials

Roeland Dubèl

Teaching the tutorial sessions

- Helps with coding exercises and questions
- Supports your progress on the group assignment
- Best person to ask during tutorials when code does not work

| r.dubel@uva.nl

The course

What is CCS-2?

- A 6 ECTS course in the Digital Society Minor
- A continuation of **CCS-1** BUT a different course!!
- Focus on applying computational methods to communication research
- Combination of:
  - concepts and theory in lectures
  - hands-on practice in tutorials

What will you learn?

By the end of this course, you should be able to:

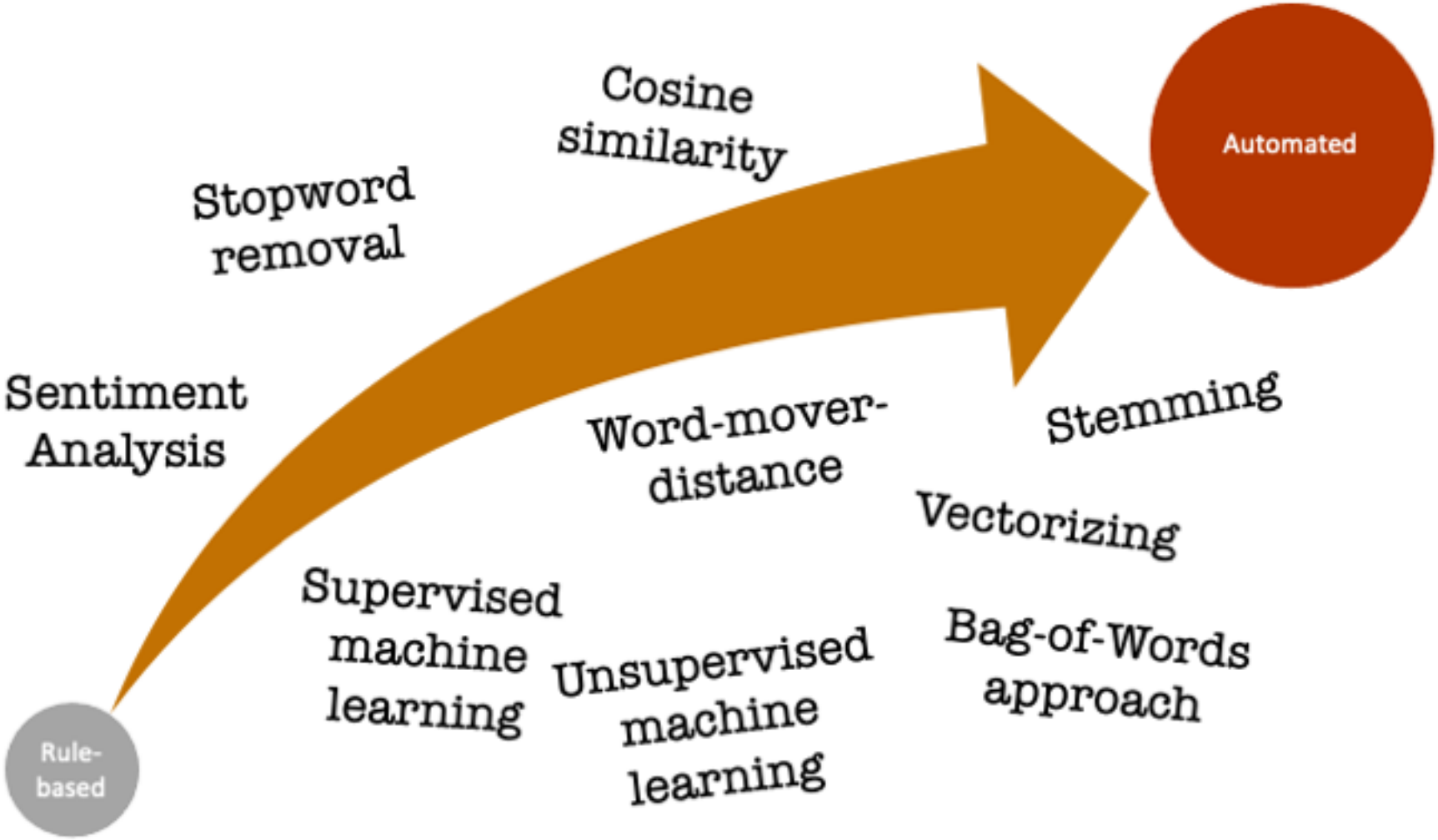
- understand key computational techniques used in communication research
- recognize benefits and limitations of different methods
- apply a subset of these techniques independently
- communicate your methodological choices clearly

What will you do in this course?

- Work with real datasets
- Analyze text and other communication data
- Explore patterns in data
- Build a simple recommender system
- Reflect critically on automated methods

In short: you will learn how to turn communication data into evidence.

Where does CCS-2 fit?



Course structure

Lectures

- Concepts, methods, examples, and implications

Tutorials

- Coding exercises
- Practice with course concepts
- Support for the group assignment

Assessment

- Multiple-choice questions (20%)
- Group assignment: written report (20%)
- Group assignment: presentation (15%)
- Exam (45%)

Assessment details

- **Multiple-choice questions**
  - 4 questions in weeks 1, 3, 4, 7, and 8
  - 20 questions in total
  - 16 correct answers = full marks
- **Group assignment**
  - Work in groups
  - Write a short report
  - Build and present a recommender system
- **Exam**
  - Final tutorial session
  - Conceptual questions + coding task

How to stay informed

Regularly check:

- The course Canvas page
- Your university email
- The course materials and announcements

Also: read the course manual carefully.

A few practical notes

- Bring a laptop with a working Python environment
- Attend lectures and tutorials regularly
- Ask coding questions during tutorials
- Use AI tools responsibly and transparently

Ready? Set? Go!

Enough logistics. . .

. . . let's get started.

Text as data

Why Computational Communication Science?

Digital communication produces large amounts of data:

- news articles
- social media posts
- online reviews
- forum discussions
- archives and documents

Computational methods help us study this data systematically and at scale.

A simple roadmap for this course

Most of what we do in this course follows the same logic:

$$\text{Text} \rightarrow \text{Data} \rightarrow \text{Representation} \rightarrow \text{Model} \rightarrow \text{Insight}$$

Today we focus mainly on how text becomes usable data.

Why do the basics still matter?

Modern AI tools can automate many language tasks.

- But the core questions remain the same:
  - How is text represented?
  - What patterns are being learned?
  - How accurate are the outputs?
  - What biases or errors may appear?

Takeaway: Understanding basic text analysis helps you evaluate modern AI tools more critically.

How computers analyze text

Raw text

↓

Preprocessing

(tokenization, cleaning, stopword removal)

↓

Text representation

(word counts, TF-IDF, embeddings)

↓

Models

Text as data

From CCS-1, you already know some basics:

- how to work with Python
- how to store and read data
- how to work with strings, lists, and dictionaries
- how to loop through data and transform it

In CCS-2, we build on that foundation.

Why study text?

Text can be useful in at least two ways:

- as the **object of study** itself
- as a **source of evidence** for a broader question

Text as data: analyzing text as a goal

Studying text can teach us a lot about communication:

- We can study what people talk about on health platforms (e.g., Sanders et al., 2020)
  - For example: which topics appear on expert-generated versus peer-generated platforms?
- We can compare language across media environments (e.g., Burggraaff and Trilling, 2020)
  - For example: how does online news differ from print news?

Text as data: analyzing text as a means

Text can also help answer broader research questions:

- We can use movie descriptions or reviews to study representation in media (e.g., Poma-Murialdo, 2019)
- We can use webpage text to distinguish between reliable and unreliable information (e.g., Meppelink et al., 2021)
- We can use content descriptions to build recommendation systems

Computational Communication Science

What does the field do?

It uses computational methods to study communication and media data.

- **Data:** news, social media, forums, archives
- **Methods:** text analysis, machine learning, network analysis
- **Goal:** generate insight from digital communication

Why it matters

It helps us study communication in natural, large-scale, digital settings.

Challenges in computational research

Key challenges

- **Data access:** what data can we actually obtain?
- **Data validity:** are these data representative?
- **Measurement:** how accurate are automated measures?
- **Ethics:** how do we use data responsibly?

Takeaway

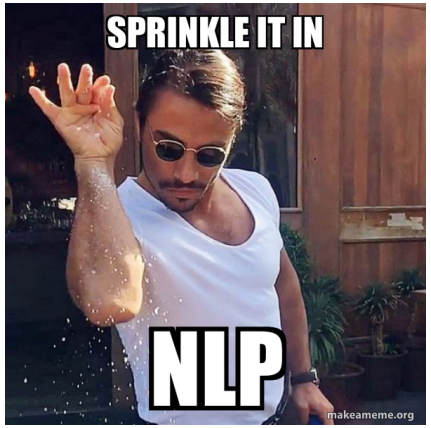
Computational methods are powerful, but they should always be used critically.

Reference: van Atteveldt and Peng, 2018

Natural Language Processing (NLP)

Definition

Natural Language Processing (NLP) studies how computers can process and analyze human language.



It often combines:

- language data
- computational rules
- machine learning

Want to know more?

Explore how large-scale text analysis changed research and public knowledge:

https://www.ted.com/talks/jean_baptiste_michel_erez_lieberman_aiden_what_we_learned_from_5_million_books

Two approaches to text analysis

Two ways to study text

Imagine you study news about climate change.

You could:

- explore the data to see which words and themes appear
- or measure the presence of predefined categories such as *policy*, *economy*, or *crisis*

These correspond to two broad approaches.

Automated content analysis often follows two main approaches:

- **Bottom-up**: exploratory, pattern-based, data-driven
- **Top-down**: guided by predefined categories or rules

Sometimes researchers also combine both.

The CCS toolbox in practice

	Methodological approach		
	Counting and Dictionary	*Supervised Machine Learning*	*Unsupervised Machine Learning*
Typical research interests and content features	visibility analysis sentiment analysis subjectivity analysis	frames topics gender bias	frames topics
Common statistical procedures	string comparisons counting	support vector machines naive Bayes	principal component analysis cluster analysis latent dirichlet allocation semantic network analysis

deductive → **inductive**

Source: Boumans and Trilling, 2016

Two approaches to automated content analysis

Bottom-up

- discover patterns in data
- no fixed categories in advance
- useful for exploration

Top-down

- use predefined categories
- measure something specific
- useful for theory-driven analysis

Understanding the difference

Bottom-up

- identify frequent words
- explore co-occurrence patterns
- ask: *What seems to be going on here?*

Key idea: we do *not* fully decide in advance what to look for.

Top-down

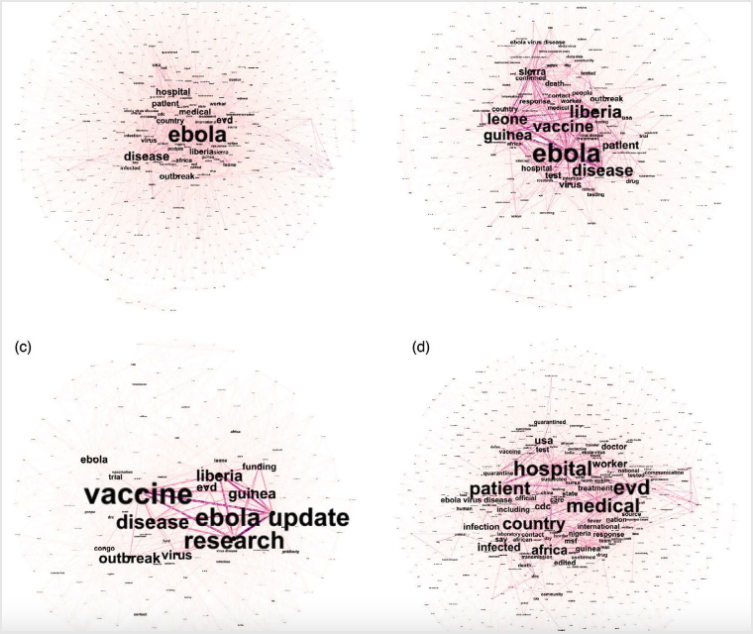
- count predefined terms
- detect known categories
- ask: *How much of X do we see?*

Key idea: we *do* decide in advance what to look for.

Research example of a bottom-up approach

Exploring prominent terms in disease-related texts

- identifies frequent words and patterns in text
- can reveal themes without predefined categories
- useful for exploratory research



You et al., 2021

Research example of a top-down approach

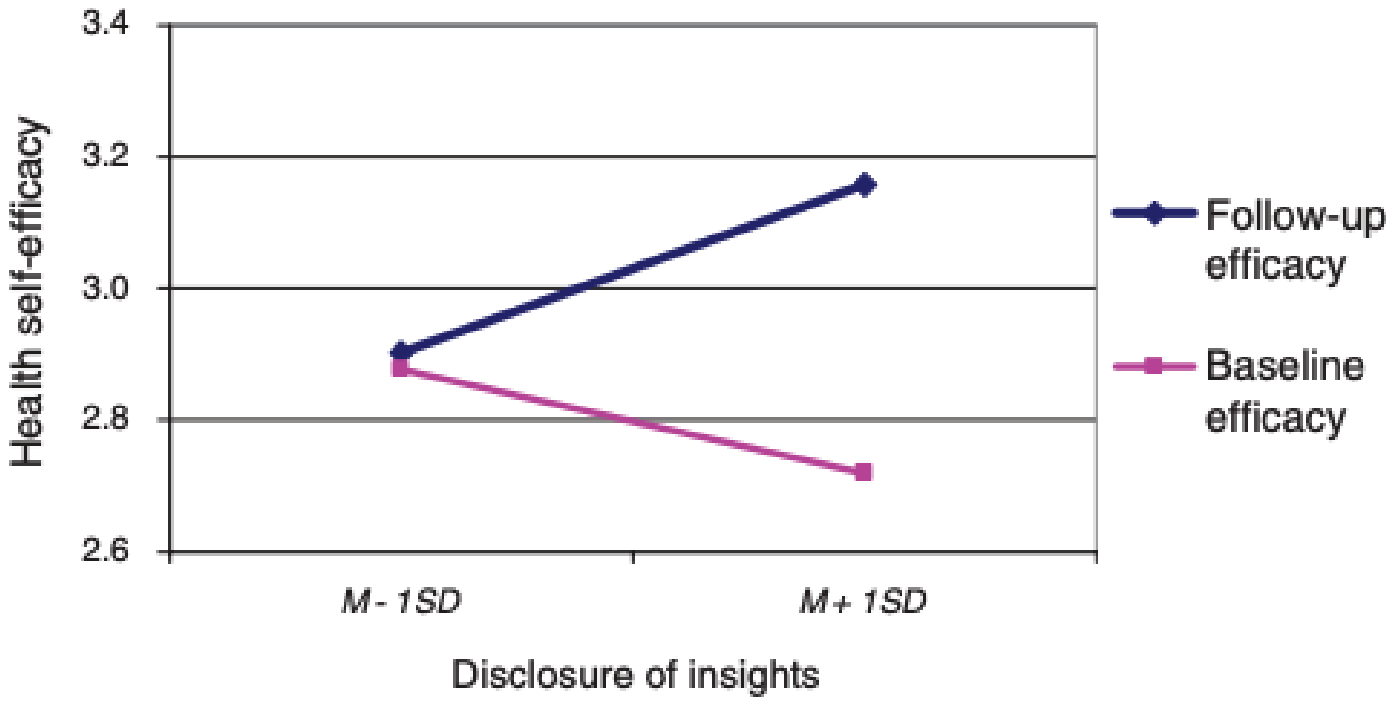
Sentiment analysis using LIWC

- LIWC uses predefined word categories
- it counts words related to emotions, cognition, and social processes
- useful when researchers want structured, theory-driven measurement

Tausczik and Pennebaker, 2010

LIWC application in online support groups

- LIWC was used to analyze language in online support groups for breast cancer patients Shim et al., 2011
- It categorizes words into emotional and cognitive dimensions
- This helped researchers connect language use to well-being outcomes



Why both approaches matter

- **Bottom-up** is useful when you want to explore
- **Top-down** is useful when you want to measure something specific
- In practice, researchers often combine both

Example: first explore which themes appear, then build a more focused measurement.

Two approaches to text analysis

Hands-on examples

A simple bottom-up approach

Goal: Identify frequent words in a small text collection.

```
from collections import Counter

texts = ["Communication in the digital society is very very complex",
                 "I like to study it"]

bottom_up = []
for text in texts:
    counts = Counter(text.lower().split())
    bottom_up.append(counts.most_common(3))

print(bottom_up)
```

This produces:

```
[[('very', 2), ('communication', 1), ('in', 1)],
 [('i', 1), ('like', 1), ('to', 1)]]
```

Note: This is a toy example that counts words within each text separately.

A simple top-down approach

Goal: Count specific words that we care about in advance.

```
texts = ["Communication in the digital society is very very complex",
                 "I like to study it"]

features = ["communication", "digital", "study"]

top_down = []
for text in texts:
    words = text.lower().split()
    counts = [words.count(feature) for feature in features]
    top_down.append(counts)

print(top_down)
```

This produces:

```
[[1, 1, 0], [0, 0, 1]]
```

When would you use which approach?

Some considerations

- Both approaches can be useful
- Bottom-up is often a good exploratory first step
- Top-down is often useful when you have a clear concept or theory
- In both cases, text must be turned into something that can be counted or compared

Preprocessing

Preprocessing text

Why preprocess?

Before we can analyze text, we usually need to transform it into a more structured format.

- This is often the first step in NLP
- It prepares text for later analysis
- We will revisit these ideas throughout the course

Preprocessing: what matters today

For now, focus on two basic questions:

- How do we split text into smaller units?
- How do we clean text so it becomes easier to analyze?

Later in the course, we will return to more advanced preprocessing choices.

Typical preprocessing steps

A basic preprocessing pipeline

Tokenization splitting text into tokens

Cleaning / filtering removing or normalizing unwanted elements

Stopword removal removing very frequent words when appropriate

Lemmatization / stemming reducing variation in word forms

Today we mainly focus on tokenization and simple cleaning.

Simple string methods

Slicing

`mystring[2:5]` gets the characters with indices 2, 3, and 4

Useful string methods

- `.lower()` returns a lowercased string
- `.strip()` removes whitespace at the beginning and end
- `.find("bla")` returns the index of a substring or -1
- `.replace("a","b")` replaces one string with another
- `.count("bla")` counts how often a substring occurs

Use tab completion to discover more.

A first simple tokenizer: `.split()`

Idea

`.split()` cuts text where there are spaces.

- easy to use
- useful for learning
- limited when punctuation and contractions matter

```
docs = ["This is a text", "I haven't seen John's derring-do. Second sentence!"]
tokens = [doc.split() for doc in docs]
tokens
```

```
[['This', 'is', 'a', 'text'],
 ['I', "haven't", 'seen', "John's", 'derring-do.', 'Second', 'sentence!']]
```

A slightly better tokenizer

Tokenizers from NLTK

Packages such as NLTK provide tokenizers that handle text more carefully.

- better with punctuation
- better with contractions

```
from nltk.tokenize import TreebankWordTokenizer

tokenizer = TreebankWordTokenizer()
tokens = [tokenizer.tokenize(doc) for doc in docs]
tokens
```

```
[['This', 'is', 'a', 'text'],
 ['I', 'have', "n't", 'seen', 'John', "'s", 'derring-do.', 'Second', 'sentence', '!']]
```

Note: Better does not mean perfect — tokenizers still make rule-based choices.

Tokenizing and removing punctuation

```
from nltk.tokenize import RegexpTokenizer

tokenizer = RegexpTokenizer(r'\w+')

text = "Hi students, what's up!"
tokens = tokenizer.tokenize(text)

print(tokens)
```

```
['Hi', 'students', 'what', 's', 'up']
```

Regex: `\w+` = one or more word characters

Note:

- Removes punctuation
- Splits contractions ("what's" → "what", "s")

Choosing a tokenizer

There is no perfect tokenizer — it depends on your goal

- `split()` → very simple (spaces only) → good for first introduction
- `TreebankWordTokenizer` → handles punctuation and contractions → good for linguistic analysis
- `RegexpTokenizer` → removes punctuation, keeps words → good for counting and simple models

Stopwords

What are stopwords?

- very frequent words with limited analytical value
- examples: `the`, `a`, `is`, `he`, `she`
- what counts as a stopword depends on your research question

For example, if you study gender, `he` and `she` may be very important.

Why remove stopwords?

- to highlight more meaningful words
- to reduce noise in comparisons between documents
- to improve some visualizations and simple analyses

But: removing stopwords is not always the right choice.

Stopword removal

```
from nltk.corpus import stopwords

mystopwords = stopwords.words("english")
mystopwords.extend(["this", "is"])

tokens_without_stopwords = [
    [word.lower() for word in doc if word.lower() not in mystopwords]
    for doc in tokens
]

tokens_without_stopwords
```

```
[['text'], ["n't", 'seen', 'John', 'derring-do.', 'Second', 'sentence', '!']]
```

A note on modern NLP tools

Many ideas we discuss today also appear in newer tools and workflows.

- `NLTK` is great for learning the basics
- Other common tools include `spaCy` and transformer-based models
- Even with newer tools, the core workflow is still:
  - tokenize
  - represent
  - model
  - evaluate

Do we still need preprocessing?

Short answer: yes — but it depends on your method

When preprocessing is important:

- Counting words (e.g., frequency, TF-IDF)
- Dictionary-based analysis (e.g., sentiment, LIWC)
- Simple models or recommender systems

Do we still need preprocessing?

When less preprocessing is needed:

- Modern AI models (e.g., BERT, GPT)
- Handle tokenization and variation internally

Key idea:

- If **you design features** → preprocessing matters
- If **model learns features** → less preprocessing

Tomorrow's tutorial

Tutorial meeting

- Discuss course goals and policies
- Revisit some CCS-1 basics
- Practice simple top-down and bottom-up text analysis
- Start thinking about the group assignment

Key takeaways

- Digital communication produces large amounts of textual data
- Computational methods help us analyze text systematically
- For most computational analysis, text must be represented in a structured form.
- Bottom-up and top-down approaches each have strengths
- Preprocessing is an important first step

Practice is key!

Thank you!

Thank you for your attention

- Questions?
- Comments?

References i

References

Boumans, J. W., & Trilling, D. (2016). **Taking stock of the toolkit: An overview of relevant autmated content analysis approaches and techniques for digital journalism scholars.** *Digital Journalism*, *4*(1), 8–23. https://doi.org/10.1080/21670811.2015.1096598

Burggraaff, C., & Trilling, D. (2020). **Through a different gate: An automated content analysis of how online news and print news differ.** *Journalism*, *21*(1), 112–129. https://doi.org/10.1177/1464884917716699

References ii

- Meppelink, C. S., Hendriks, H., Trilling, D., van Weert, J. C., Shao, A., & Smit, E. S. (2021). **Reliable or not? an automated classification of webpages about early childhood vaccination using supervised machine learning.** *Patient Education and Counseling*, *104*(6), 1460–1466. https://doi.org/10.1016/j.pec.2020.11.013
- Poma-Murialdo, S. C. (2019). ***Gender inequality in the movie industry:* [Master's Thesis]. University of Amsterdam.**
- Sanders, R., Linn, A. J., Araujo, T. B., Vliegenthart, R., van Eenbergen, M. C., & van Weert, J. C. (2020). **Different platforms for different patients' needs: Automatic content analysis of different online health information platforms.** *International Journal of Human-Computer Studies*, *137*, 102386. https://doi.org/10.1016/j.ijhcs.2019.102386

References iii

- Shim, M., Cappella, J. N., & Han, J. Y. (2011). **How does insightful and emotional disclosure bring potential health benefits? study based on online support groups for women with breast cancer.** *Journal of Communication*, *61*(3), 432–454.
- Tausczik, Y. R., & Pennebaker, J. W. (2010). **The psychological meaning of words: Liwc and computerized text analysis methods.** *Journal of language and social psychology*, *29*(1), 24–54.
- van Atteveldt, W., & Peng, T.-Q. (2018). **When communication meets computation: Opportunities, challenges, and pitfalls in computational communication science.** *Communication Methods and Measures*, *12*(2-3), 81–92. https://doi.org/10.1080/19312458.2018.1458084

References iv

You, J., Expert, P., & Costelloe, C. (2021). **Using text mining to track outbreak trends in global surveillance of emerging diseases: Promed-mail.** *Journal of the Royal Statistical Society Series A: Statistics in Society*, *184*(4), 1245–1259.